

Báo Cáo Bài Tập Thực Hành MCLab01

1. Thông tin sinh viên

- **MSSV:** 23110156
- **Họ tên:** Nguyễn Minh Hoàng
- **Lựa chọn bài toán:** A - Task Pocket - Quản lý công việc cá nhân

2. Problem & Requirements

- **Mô tả ngắn bài toán:** Xây dựng phần logic cho một ứng dụng điện thoại quản lý công việc cá nhân. Chưa có giao diện; người dùng được mô phỏng bằng các thao tác gọi hàm trong index.ts và xem kết quả trên Terminal. Với ứng dụng, người dùng có thể quản lý danh sách các công việc cá nhân. Mỗi công việc sẽ có mã số, tên, số giờ dự kiến sẽ làm công việc đó, trạng thái, mức độ ưu tiên, và có thể có người phụ trách. Chương trình sẽ hiển thị dữ liệu output qua console, lọc theo trạng thái, tính tổng số giờ dự kiến, tìm công việc theo ID và xử lý các tình huống dữ liệu không đầy đủ.
- **Các chức năng đã làm:**

- **Các chức năng bắt buộc:**

Chức năng logic	Output quan sát trong Terminal	Khái niệm TypeScript
Danh sách Task	In từng công việc	Array of objects + type alias
Task status	todo/doing/done	Literal union
Assignee có thể thiếu	"Chưa phân công" khi chưa có	Optional/Nullable + ??
FilterByStatus	Danh sách theo trạng thái công việc	Typed function
Tổng số giờ dự kiến	Tổng số giờ theo filter	number return type
Tìm task theo ID	Task hoặc null	Union + narrowing
Action/callback	Mô phỏng mở Task	Function type
Raw data validation	Reject dữ liệu sai	unknown + type guard

- **Các chức năng mở rộng thêm:**

Chức năng logic	Output quan sát trong Terminal	Khái niệm TypeScript
FilterByPriority	Danh sách theo độ ưu tiên công việc	Typed function
SortTasks	Sắp xếp danh sách công việc theo thời gian dự kiến hoặc độ ưu tiên (giảm dần hoặc tăng dần)	Typed function

```
TinDev > mc-typescript-lab > 0ms npm init -y
Wrote to D:\MobileComputing\Lab01\mc-typescript-lab\package.json:

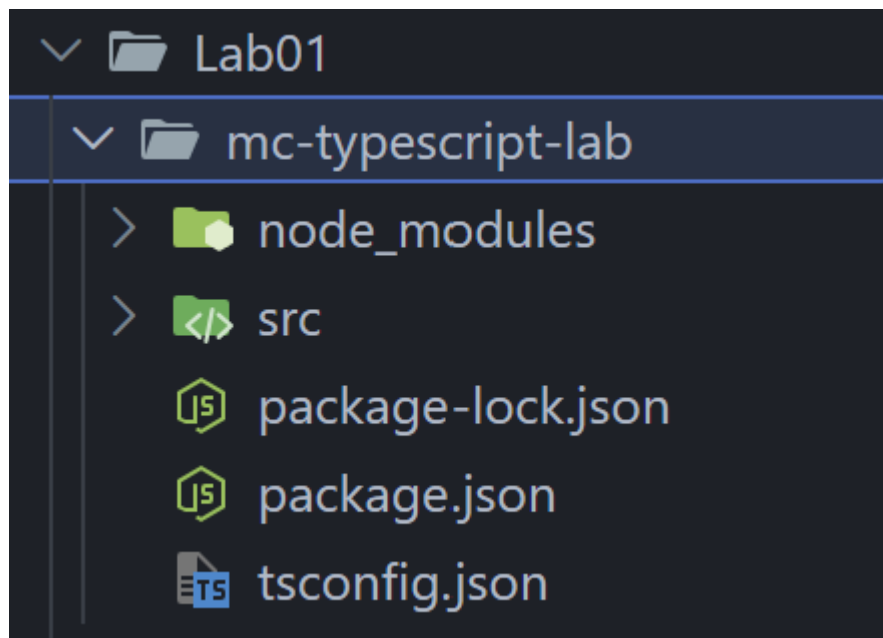
{
  "name": "mc-typescript-lab",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
TinDev > mc-typescript-lab > 24.8.0 > 14.953s npm install --save-dev typescript
added 21 packages, and audited 22 packages in 3s

found 0 vulnerabilities
TinDev > mc-typescript-lab > 24.8.0 > 3.887s npx tsc --init
Created a new tsconfig.json
You can learn more at https://aka.ms/tsconfig
TinDev > mc-typescript-lab > 24.8.0 > 1.903s mkdir src

Directory: D:\MobileComputing\Lab01\mc-typescript-lab

Mode                LastWriteTime         Length Name
----                -
d-----          10/5/2026   4:15 PM             src
```



3. Type Model

Toàn bộ kiểu dữ liệu được định nghĩa trong `task.types.ts`:

```
export type TaskStatus = "todo" | "doing" | "done";
export type TaskPriority = "low" | "medium" | "high";
export type TaskFilter = "all" | TaskStatus | TaskPriority;
export type SortBy = "estimatedHours" | "priority";
export type SortOrder = "asc" | "desc";

export type Task = {
  id: string;
  title: string;
  estimatedHours: number;
  status: TaskStatus;
  priority: TaskPriority;
  assignee: string | null; //nullable property
  note?: string; //optional property
};
```

Giải thích các khái niệm TypeScript sử dụng:

Khái niệm	Áp dụng	Ý nghĩa
Type Alias	<code>type Task = { ... }</code>	Đặt tên cho blueprint của đối tượng
Literal Union	<code>"todo" "doing" "done"</code>	Giới hạn giá trị chỉ trong tập hợp cố định
Nullable	<code>string null</code>	Trường bắt buộc tồn tại nhưng có thể rỗng
Optional (<code>?:</code>)	<code>note?: string</code>	Trường có thể không tồn tại trong object

4. Logic chính

Toàn bộ logic nghiệp vụ nằm trong `task.service.ts` và `task.callback.ts`:

`getTasks(tasksList)` — Lấy và hiển thị danh sách Task

Duyệt từng task, áp dụng **Nullish Coalescing (??)** để fallback assignee về "Unassigned" khi giá trị là `null`.

`filterTasksByStatus(tasksList, filter)` — Lọc theo trạng thái

Nhận tham số kiểu `TaskFilter` (union type), trả về `Task[]`. Nếu filter là "all" thì trả về toàn bộ danh sách.

`filterTasksByPriority(tasksList, filter)` — Lọc theo mức ưu tiên

Tương tự filter status, sử dụng cùng kiểu `TaskFilter` nhờ union mở rộng "all" | `TaskStatus` | `TaskPriority`.

`calculateTotalEstimatedHours(tasksList)` — Tính tổng số giờ

Dùng `Array.reduce()` để cộng dồn `estimatedHours`, return type tường minh là `number`.

`findTaskById(tasksList, taskId)` — Tìm task theo ID (Safety pattern)

Return type là `Task` | `null`. Caller **bắt buộc phải narrowing** trước khi sử dụng kết quả:

```
const task = findTaskById(tasks, "MC002");
if (task) {
  console.log(task.title);
}
```

`isTask(value)` — Custom Type Guard (External data validation)

Nhận `value: unknown` (dữ liệu bên ngoài chưa tin cậy), kiểm tra từng trường, return `value is Task`. Sau khi guard trả về `true`, TypeScript tự động narrow kiểu:

```
if (isTask(unknownValue)) {
  console.log(unknownValue.title); // ✅ TypeScript biết đây là Task
}
```

`sortTasks(tasksList, sortBy, sortOrder)` — Sắp xếp

- **Không mutate** mảng gốc: dùng `[...tasksList]` để copy trước khi sort.
- Sort `estimatedHours`: phép trừ số học trực tiếp.
- Sort `priority`: dùng `indexOf()` để map chuỗi sang số thứ tự (`low=0`, `medium=1`, `high=2`).
- Dùng `direction = sortOrder === "asc" ? 1 : -1` để tránh duplicate logic.

`openTask(taskId)` — Callback pattern (`task.callback.ts`)

Định nghĩa type `OnOpenTask = (taskId: string) => void` làm blueprint, sau đó implement bằng `export const openTask: OnOpenTask = ....` Dùng `findTaskById` kết hợp narrowing để xử lý an toàn.

5. Kết quả chạy

Biên dịch và chạy bằng lệnh:

```
npx tsc
node dist/index.js
```

Output:

Welcome to Task Pocket!

All tasks:

Task ID: MC001 | Title: Doing lab1 mobile computing | Estimated Hours: 8 | Status: doing | Priority: high | Assignee: Unassigned
Task ID: MC002 | Title: Building realtime chat app | Estimated Hours: 5 | Status: todo | Priority: medium | Assignee: Unassigned
Task ID: MC003 | Title: Write Unit Test for the wallet module | Estimated Hours: 1 | Status: done | Priority: low | Assignee: Hoang
Task ID: MC004 | Title: Implementing the new payment gateway | Estimated Hours: 10 | Status: doing | Priority: high | Assignee: Unassigned

Doing tasks:

Task ID: MC001 | Title: Doing lab1 mobile computing | Estimated Hours: 8 | Status: doing | Priority: high | Assignee: Unassigned
Task ID: MC004 | Title: Implementing the new payment gateway | Estimated Hours: 10 | Status: doing | Priority: high | Assignee: Unassigned

Total estimated hours for all tasks: 24

Task with ID MC002: Title: Building realtime chat app | Estimated Hours: 5 | Status: todo | Priority: medium | Assignee: Unassigned

Task with ID MC003 and title "Write Unit Test for the wallet module" has been opened.

Valid task: Integrate AI features

High priority tasks:

Task ID: MC001 | Title: Doing lab1 mobile computing | Estimated Hours: 8 | Status: doing | Priority: high | Assignee: Unassigned
Task ID: MC004 | Title: Implementing the new payment gateway | Estimated Hours: 10 | Status: doing | Priority: high | Assignee: Unassigned

Tasks sorted by estimated hours (ascending):

Task ID: MC003 | Title: Write Unit Test for the wallet module | Estimated Hours: 1 | Status: done | Priority: low | Assignee: Hoang
Task ID: MC002 | Title: Building realtime chat app | Estimated Hours: 5 | Status: todo | Priority: medium | Assignee: Unassigned
Task ID: MC001 | Title: Doing lab1 mobile computing | Estimated Hours: 8 | Status: doing | Priority: high | Assignee: Unassigned
Task ID: MC004 | Title: Implementing the new payment gateway | Estimated Hours: 10 | Status: doing | Priority: high | Assignee: Unassigned

Tasks sorted by priority (descending):

Task ID: MC001 | Title: Doing lab1 mobile computing | Estimated Hours: 8 | Status: doing | Priority: high | Assignee: Unassigned
 Task ID: MC004 | Title: Implementing the new payment gateway | Estimated Hours: 10 | Status: doing | Priority: high | Assignee: Unassigned
 Task ID: MC002 | Title: Building realtime chat app | Estimated Hours: 5 | Status: todo | Priority: medium | Assignee: Unassigned
 Task ID: MC003 | Title: Write Unit Test for the wallet module | Estimated Hours: 1 | Status: done | Priority: low | Assignee: Hoang

6. Code Organization

Dự án được tổ chức theo nguyên tắc **Separation of Concerns** — mỗi file có một trách nhiệm rõ ràng:

```
mc-typescript-lab/
├─ tsconfig.json      # Cấu hình TypeScript compiler (target ES2025, strict mode)
├─ package.json       # Dependencies (typescript devDependency)
├─ src/
│   ├─ task.types.ts  # Tầng Type – toàn bộ blueprint kiểu dữ liệu
│   ├─ task.data.ts   # Tầng Data – mock data (4 task mẫu)
│   ├─ task.service.ts # Tầng Service – logic nghiệp vụ (6 hàm)
│   ├─ task.callback.ts # Tầng Callback – event handler pattern
│   └─ index.ts       # Entry point – mô phỏng tương tác người dùng
└─ dist/             # Output JavaScript sau khi biên dịch (git ignored)
```

Lưu đồ phụ thuộc (Dependency Flow):

```
task.types.ts ← task.data.ts
task.types.ts ← task.service.ts
task.service.ts ← task.callback.ts
task.data.ts ← task.callback.ts
task.service.ts ← index.ts
task.data.ts ← index.ts
task.callback.ts ← index.ts
```

`task.types.ts` là nền tảng — không import từ file nào khác, toàn bộ file còn lại import từ nó.

7. Git/Github/CI

<https://github.com/tin1508/MobileComputing/tree/main/Lab01>

8. Problem Solving

9. Interview Questions

10. Self-check

